

GarageFlow API and Endpoint Structure Design

Understanding the System:

Actions	System Reactions
Vehicles enter	Creating tickets, recording relevant customer information
Vehicles exit	Closing tickets, calculating fees, deleting leftover customer data
Parking spaces	Assigning, freeing, checking availability status
Reservations	Booking future spots
Capacity	Real-time occupancy data for operator
Analytics	Occupancy reports, utilization data
Security	Authentication, permissions

- REST API Conventions:

Endpoints will follow REST conventions. The pattern is:

METHOD /resource/{id}

The four methods I will be utilizing are:

1. GET // Fetch data
2. POST // Create something new
3. PUT // Update something existing
4. DELETE // Remove something

- Base URL & Example Endpoints:

- <https://api.GarageFlow.com>
- <https://api.GarageFlow.com/v1/tickets>
- <https://api.GarageFlow.com/v1/spaces>
- <https://api.GarageFlow.com/v1/reservations>

- API Architecture

GarageFlow follows a REST architecture using JSON over HTTPS. All endpoints are versioned under `/v1/` to allow future iterations without breaking existing integrations.

- Authentication & Roles

All endpoints except payment webhooks require a Bearer token obtained via `POST /v1/auth/login`. Tokens expire and must be refreshed using `POST /v1/auth/refresh`. Three roles exist in the system:

- *Attendant* — can create and close tickets, look up reservations, and process payments
- *Admin* — all attendant permissions plus user management, audit log access, and payment overrides
- *Super-admin* — all admin permissions plus hard deletion of users and audit records

- Error Handling

Common HTTP endpoint status codes across the API:

- 200 — Success
- 201 — Resource created
- 400 — Missing or invalid fields
- 401 — Unauthenticated
- 403 — Insufficient permissions
- 404 — Resource not found
- 409 — Conflict (e.g. garage full, plate mismatch)
- 429 — Too many requests
- 500 — Server error

```
{  
  "error": "plate_mismatch",  
  "message": "License plate does not match reservation."  
}
```

}

- Data Retention Policy

Personal data (phone number, license plate) is handled according to a tiered purge schedule. Upon reservation cancellation, personal data is removed from the reservation record immediately. Upon ticket completion, personal data is removed from the reservation record immediately. An anonymous record is retained for analytics. On ticket close, personal data is retained until the end-of-day purge following vehicle exit, giving staff a short window to resolve gate disputes. This purge is automated and does not rely on human follow-through.

- Payment Processing

GarageFlow uses Stripe as its payment processor. The API accepts Stripe payment intent tokens only. Raw card data is never transmitted through or stored by GarageFlow. Only a phone number is required from the customer for payment and SMS notification purposes.

- Audit Logging

All state-changing actions (ticket creation and closure, reservation modifications, payment overrides, user role changes) are written to an immutable audit log accessible at `/v1/admin/history`. Every log entry records the acting user ID, action type, timestamp, and affected resource. Audit records cannot be deleted by any role except super-admin.

- Rate Limiting

Rate limits protect the API from abuse and ensure fair resource distribution across clients. Limits are enforced per Bearer token and communicated through standard response headers.

X-RateLimit-Limit: 200

X-RateLimit-Remianing: 147

X-RateLimit-Reset: 1741694400 // Unix timestamp of when the window resets
Limits by Role:

Role	Requests / minute	Notes
Operator	120	Covers regular operations
Admin	200	Higher limit for report generation
Super-admin	300	Unrestricted for audit and maintenance tasks
Stripe webhook	Infinity	Exempt because Stripe controls its own retry logic

Endpoint List

1. Tickets

Method	Endpoint	Description
POST	/v1/tickets	Car arrives, create ticket
GET	/v1/tickets/{id}	Look up a ticket by ID
GET	/v1/tickets	List all active tickets
GET	/v1/tickets?plate=ABC-123	Look up a ticket by plate
GET	/v1/tickets?phone=555-0100	Look up ticket by phone number
PUT	/v1/tickets/{id}/exit	Car leaves, close ticket. Needs to trigger or confirm payment before closing the ticket
DELETE	/v1/tickets/{id}/personal	Wipes name, phone, and plate from that ticket

DELETE	/v1/tickets/{id}	Full ticket removal, admin only
--------	------------------	---------------------------------

Tickets are identified by a unique system-generated ticket ID. The phone number is stored on the ticket as a secondary lookup field and is used for SMS notifications. Multiple active tickets may share a phone number; in this case, lookup by phone returns all matches and staff disambiguates by license plate.

An edge case that has been realized and resolved is the following:

- What happens when a phone number has two active tickets (two family cars)?
- The solution is to allow it, requiring plate confirmation — the system asks for a license plate upon exit to disambiguate. This feature also functions as an additional physical security check.

2. Parking Spaces

GET	/v1/spaces	List all spaces and their details
GET	/v1/spaces/available	List all available spaces and their details
GET	/v1/spaces/available?type=standard	Available by parking space type (employee/accessibility/eco) on any floor
GET	/v1/floors/{floorId}/spaces	All spaces on a specific floor
GET	/v1/floors/{floorId}/spaces?status=available	Available spaces on a specific floor
GET	/v1/floors/{floorId}/spaces/{spaceId}	One specific space
PUT	/v1/floors/{floorId}/spaces/{spaceId}	Manually update a specific space (random hazards, etc.)

POST	/v1/floors	Add a new floor
GET	/v1/floors	List all floors and respective capacities
DELETE	/v1/floors/{floorId}	Delete a floor
PUT	/v1/floors/{floorId}	Edit floor settings (label, capacity, space breakdown)
GET	/v1/floors/{floorId}	Get a floor's details and capacity
POST	/v1/floors/{floorId}/spaces	Add a single space to a specific floor
DELETE	/v1/floors/{floorId}/spaces/{spaceId}	Delete a single space on a specific floor

Spaces are managed through floor-scoped endpoints (/v1/floors/{floorId}/spaces) for all administrative and floor-level operations. A separate /v1/spaces/available endpoint exists for garage-wide availability queries used by the automatic space assignment logic. Each space object always includes its floor ID and label.

- Assignment Algorithm (server-side):
 - Filter available spaces matching the requested driver class (eco, handicap, employee)
 - Within matches, prefer spaces on the lowest available floor to minimize driver travel time.
 - Distributes customers across floors to reduce congestion.

3. Capacity

GET	/v1/capacity	Live occupancy vs total
GET	/v1/capacity/status	Does a full live

		availability check
GET	/v1/capacity/floors/{floorId}	Live capacity on one specific floor

4. Reservations

POST	/v1/reservations	Book a future space
GET	/v1/reservations	List all reservations
GET	/v1/reservations?plate=ABC123	Search reservations under a license plate
GET	/v1/reservations/{id}	Get reservation details
PUT	/v1/reservations/{id}	Modify an existing reservation
PUT	/v1/reservations/{id}/check	Convert reservation to active ticket on arrival, license plate validation needed
DELETE	/v1/reservations/{id}	Cancel or delete an existing reservation, retains record
GET	/v1/reservations?phone=555-0100	Look up by phone number

A practical setup is that the monitor operator at entry can handle reservations as a part of their existing role. Someone calls ahead, the attendant creates the reservation through the same interface, and then waits for their arrival.

An edge case that has been realized and resolved is the following:

- If someone calls to cancel their reservation and they are not who they claim to be.
- Operators must verify the license plate and phone number listed in the reservation to cancel an existing reservation.

An edge case that has been realized and resolved is the following:

- A driver is late to their reservation.
- The system does not automatically expire the reservation. The operator must use their discretion to decide whether or not the reservation should be expired.

PUT /v1/reservations/{id}/check converts a reservation into an active ticket on arrival. This is the most complex flow in the system because it spans two resources (reservations and tickets), requires identity verification, triggers personal data purge, and must handle timing edge cases.

- "licensePlate": "ABC123" // is the only required field.
- Reservation Check-In Flow
 1. Look up the reservation by ID or phone. If not found → 404.
 2. Confirm reservation status is confirmed, not already checked in or cancelled. If invalid → 409.
 3. Provide the license plate of the vehicle to the system, If not provided → 409 plate_mismatch.
 4. The system updates the status of reserved parking occupancy reservation to that of a ticket, with a link to the reservation ID.
 5. Return the new ticket object.

5. Payment

POST	/v1/payments	Charge a ticket or reservation
GET	/v1/payments/{id}	Get payment details
GET	/v1/payments?ticketId={id}	Look up payment by ticket

GET	/v1/payments?plate=ABC123	Look up payment history by plate
POST	/v1/payments/{id}/refund	Issue a full or partial refund after a manual step
GET	/v1/payments/reports	Revenue summary for admin
POST	/v1/payments/{id}/override	Attendant override with admin permissions

GarageFlow utilizes Stripe as its payment processor. All payment endpoints accept Stripe payment intent tokens. Raw card data is never stored or transmitted through the GarageFlow API.

6. Analytics

GET	/v1/analytics/utilization	Utilization rate over time
GET	/v1/analytics/occupancy	Current and long-term historical occupancy
GET	/v1/analytics/peak-hours	Peak usage time analysis

7. Account Endpoints

POST	/v1/auth/login	Login, returns token
POST	/v1/auth/refresh	For token refresh
POST	/v1/auth/logout	Invalidate session
GET	/v1/auth/me	Get the current user's permissions
POST	/v1/auth/register	Create a new user
PUT	/v1/auth/me/password	Change own password

8. Admin Management

GET	/v1/users	List of users
GET	/v1/users/{userId}	Get a user's details
DELETE	/v1/users/{userId}	Delete a user
PUT	/v1/users/{userId}/role	Change a user's permissions
PUT	/v1/users/{userId}/password	Change a user's password
GET	/v1/admin/history	Full audit log
GET	/v1/admin/history?userId={id}	One user's history
GET	/v1/admin/history?action=ticket.close	Filter by action type
GET	/v1/admin/history?from=2026-03-01	Filter by date range
PUT	/v1/users/{userId}/status	Set an account to active or deactivated

Administrators utilize PUT /v1/users/{userId}/status to deactivate an account, meaning it is disabled but the user's audit history remains intact and readable. DELETE /v1/users/{userId} is a hard delete restricted to super-admin, used only for miscellaneous account creation. Deleting a user with existing audit log entries is not permitted to anybody except the super-admin.

9. Stripe

POST	/v1/webhooks/stripe	Receive Stripe event notifications
------	---------------------	------------------------------------

This endpoint does not require a Bearer token. Instead, every incoming request is verified using Stripe's webhook signature. This is a header that GarageFlow validates against the shared webhook secret. Requests that fail signature validation are rejected immediately with 401. This is the standard Stripe security model.

An edge case that has been realized and resolved is the following:

- Stripe may deliver the same event more than once.
- GarageFlow deduplicates by storing the stripeEventId upon receiving the first receipt and then discards subsequent deliveries of the same ID. All webhook handlers respond with 200 immediately upon receipt, as processing happens asynchronously. These methods prevent Stripe from retrying due to slow processing time.

Example

POST /tickets

Description: Creates a new parking ticket when a vehicle enters the garage

Request Body:

```
{
  "licensePlate": "ABC123",
  "vehicleType": "compact" | "standard" | "oversized",
  "entryGateId": "GATE-01"
  "phone": "555-0199"           // optional - walk-up customers may omit.
}
```

Success Response (201 Created):

```
{
  "ticketId": "T-0042",
  "licensePlate": "ABC123",
  "phone": "555-0199"
```

```
"assignedSpace": "B-12",  
"entryTime": "2026-03-11T09:15:00Z",  
"status": "active"  
}
```

Error Responses:

- 400 — Missing required fields
- 409 — No spaces available (garage full)
- 500 — Server error